CDT Healthcare Data Science

ML for Time Series Module

Wearable Laboratory 1: From ECG Signals to Features and a Classifier

Organiser(s): ..... [Junjie Chen]



Department of Engineering Science

University of Oxford

Nov 2026

Table of contents

0 Lab overview 1
1 What is an ECG signal? 2
2 Measuring rhythm from ECG signals 5
3 Extracting rhythm features from noisy raw signals 8
3.1 NeuroKit2: a reference beat detector 9
3.2 A naive peak-finding detector 10
3.3 A beat detector that searches for the QRS complex 11
3.4 Median imputation: correcting the HRV estimate 16
4 Classifying rhythms with ECG features 19
4.1 NeuroKit2 ECG features 19
4.2 Training a RandomForest with the features 20
Submitting your work 22

Bibliography **23**

0 Lab overview

This lab takes a raw electrocardiogram (ECG), turns it into a few numbers that describe the heart's rhythm, and uses such numbers to classify the rhythm. It is organised around four questions:

Question	Your work	Notebook
1 What is an ECG signal?	Label the P, Q, R, S and T dots on five excerpts	1
2 What do people measure from it?	Code heart rate, SDRR and RMSSD from NeuroKit2's RR sequences	2
3 Can we extract them ourselves?	Code a naive peak-finding detector, then a QRS-searching detector in three steps and median imputation, each scored against NeuroKit2	3, 4, 4A–4D
4 What are the features for?	Code a RandomForest on the full 54 NeuroKit2 features, on Lab 2's split	5

In this lab, we will use known-good ECG features. In Lab 2, we will turn to end-to-end methods, removing the dependency to searching for good ECG features.

How to work. Open `ECG_Preprocessing.ipynb` and run the supplied cells in order. Each task is a function whose body has been left empty for you to write. Its specification is in this note, in a box that states what you are given, what you may use, what to return and how it is checked. The notebook runs your function against a reference and tells you whether it agrees; an unfinished task prints a reminder and falls back to the reference, so the later cells still run.

Setup. The lab lives in a GitHub repository (https://github.com/junjiechen-chris/CDT-HDS-ECG-Lab-1). Open the notebook in Colab using the badge in its first cell and save a copy to your Drive. You do not need to install anything yourself: the Setup cell fetches the code, installs the packages the lab depends on, mainly NeuroKit2 [1] and scikit-learn, and downloads the data. The data are password-protected, and the Setup cell will prompt you for the password. It will be handed out during the lab, so ask your tutor. No GPU is needed. Once the Setup cell has run, the tasks are all that is left for you to do.

1 What is an ECG signal?

Notebook: Section 1

An electrocardiogram records the voltage between two electrodes placed on the skin, sampled many times a second. Every heartbeat leaves the same short pattern in that voltage: a P wave, then a QRS complex, then a T wave (Figure 1, top). The sharp R peak at the top of the QRS complex tells us *when* the beat happened, and everything else in this lab is built from those moments in time. In recordings from a wearable device, however, the pattern rarely arrives clean. It often sits on a drifting baseline, is buried in noise, or appears upside down (Figure 1, bottom). This is why we cannot simply look for the highest points in the signal: we need a beat detector that copes with all three.

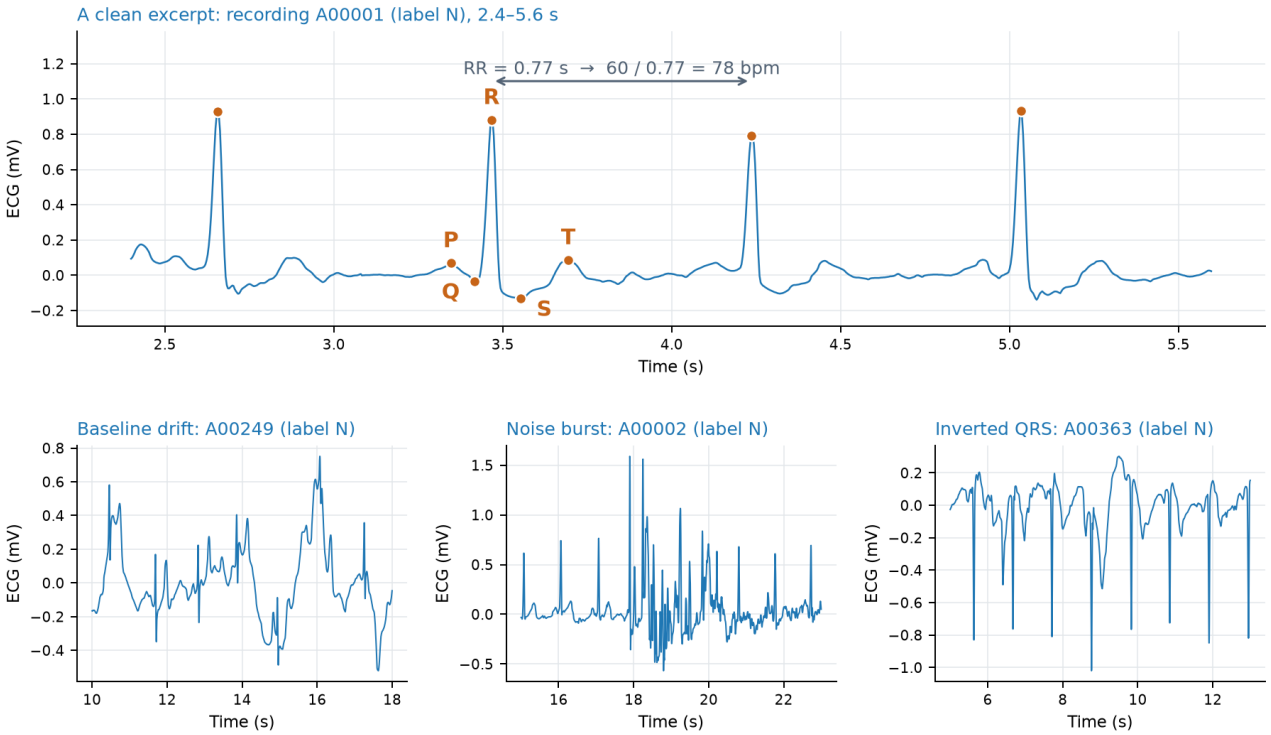


Figure 1: Top: three beats from a clean excerpt. One beat has its P, Q, R, S and T waves marked, and the RR interval is shown between two R peaks. Bottom: three things that get in the way, each taken from a recording labelled `N`: baseline drift, a burst of noise, and a QRS complex that points downwards.

The waves of one beat. Each wave corresponds to one stage of the heartbeat. The **P wave** is small and rounded, and marks the atria contracting. The **QRS complex** is a sharp spike, roughly 80 ms to 120 ms wide and usually the tallest feature of the beat; it marks the ventricles contracting. The Q and S waves are the small dips on either side of the R peak. The **T wave** is broad and rounded, and marks the ventricles recovering. In a normal beat these waves always

appear in the same order, so their positions relative to the R peak hardly vary. The wave-shape features used in Section 4 measure exactly those positions and heights.

Timing. In the top panel, the two marked R peaks are 0.77 s apart. That gap is one **RR interval**, and it corresponds to a heart rate of $60/0.77 = 78$ beats per minute. At rest, RR intervals are usually between 0.6 s to 1.0 s, which is 60 bpm to 100 bpm. The RR interval is the first feature of this lab, and it is one you can read directly off the plot.

Three things that get in the way. *Baseline drift* (bottom left) comes from breathing, body movement and changing electrode contact. It is slow, below about 0.5 Hz, and the beats ride up and down on top of it, so a high voltage no longer means a beat. A *noise burst* (bottom middle) comes from movement or muscle activity. For a few seconds the noise spikes are as tall as the R peaks. Notice that this recording is still labelled `N`: the label describes the rhythm, not the quality of the signal. An *inverted QRS* (bottom right) depends on where the electrodes sit on the body. Here the QRS complex points downwards, so the maximum of the signal is never the beat. Each of these would defeat a naive rule, and Section 3 has one step to deal with each of them.

The recordings. The data come from the PhysioNet/Computing in Cardiology Challenge 2017 [2, 3] and are for educational use only.

Item	Value
Device	a handheld single-lead recorder, two electrodes under the fingers
Recordings	2,000: `N` 1,187, `A` 196, `O` 553, `~` 64
Length	9 to 60 seconds
Sampling rate	300 Hz: time in seconds = sample index / 300
Units	stored in microvolts; the notebook divides by 1,000 to plot millivolts
Labels	one of four rhythm classes per recording, assigned by experts: `N`, `A`, `O` or `~`, explained below

Four heart rhythm classes. The heart has its own pacemaker, the sinus node, which fires at a steady rate and triggers every normal beat. The four labels describe how far a recording's rhythm departs from that (Figure 2):

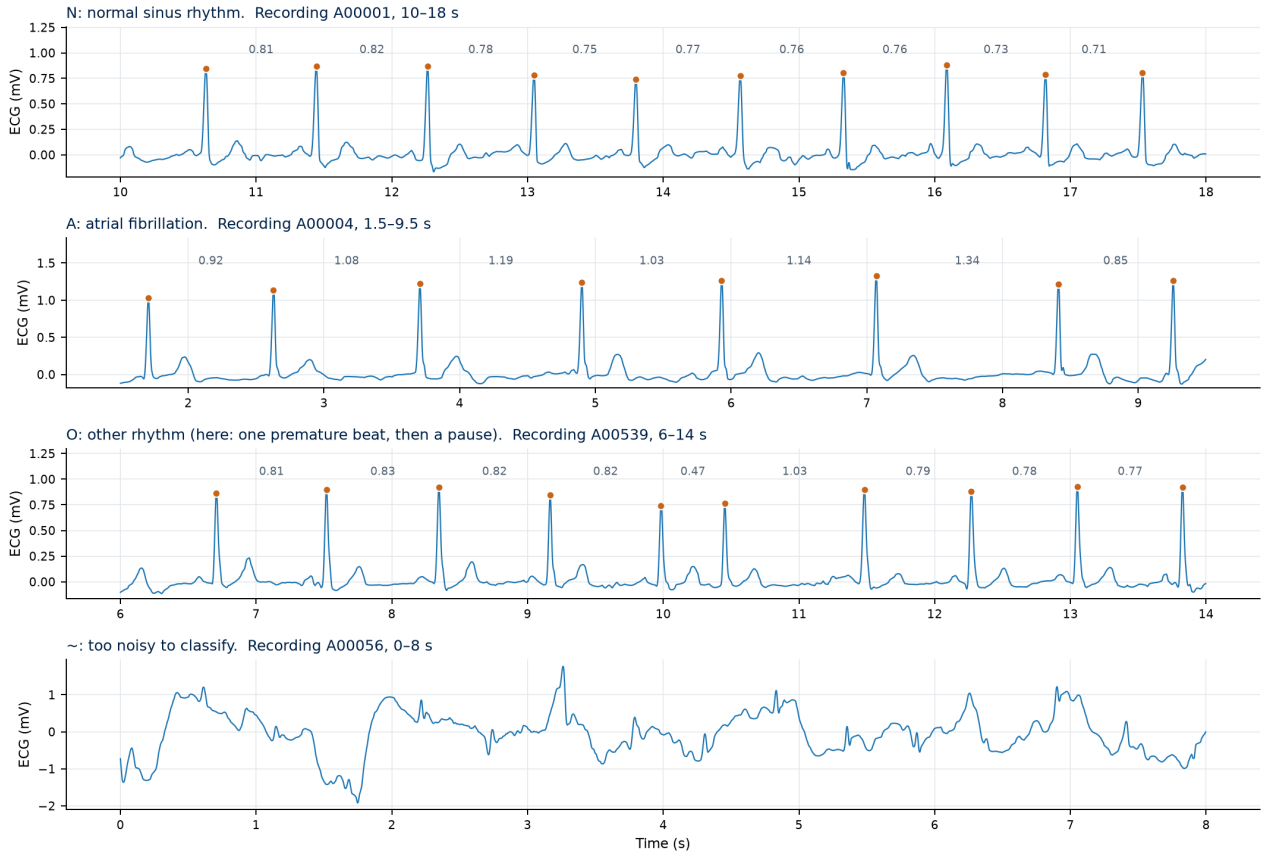


Figure 2: One recording from each class, 8 seconds each. The dots mark the beats and the numbers above them give the RR intervals in seconds. `N`: evenly spaced beats, each with a P wave. `A`: intervals that change from one beat to the next, and no P waves. `O`: a regular rhythm interrupted by one early beat and a pause. `~`: no beat can be recognised at all.

- **`N`, normal sinus rhythm.** The sinus node drives every beat. Each beat shows a P wave, a QRS complex and a T wave in turn, and the RR intervals change only gradually, for example with breathing. On the plot, the beats are evenly spaced and all look alike.

- **`A`, atrial fibrillation (AF).** The atria quiver instead of contracting, so there are no P waves, only a wavy baseline. The ventricles are triggered at random moments, so the RR intervals jump from one beat to the next: the rhythm is *irregularly irregular*. The QRS complex itself looks normal, because the ventricles still contract normally. AF is the most common sustained arrhythmia and raises the risk of stroke, which is why being able to detect it from a wearable matters.

- **`O`, other rhythm.** Everything that is neither normal nor AF. This includes a premature beat that arrives early and is followed by a pause (as in the figure), runs of fast beats, a slow rhythm, and blocked conduction. It is a mixed group with no single appearance, which is what makes it the hardest class in Section 4.

- **~, too noisy to classify.** The signal is so corrupted that no rhythm can be read from it.

A label describes the rhythm of the *whole* recording. Apart from ~, it says nothing about the quality of the signal, and it does not promise that every beat is normal: an N recording can still contain an artefact or a single premature beat.

Task 1. The notebook shows you six excerpts from N recordings. In each one, ten consecutive wave points spanning two beats are marked with numbered dots. The first excerpt is labelled for you as the worked example (P Q R S T P Q R S T). The other five each start at a different point in the cycle, so you have to read each dot from the shape of the signal rather than from its position in the sequence. *Submit:* the ten letters for each of the five diagrams, in order, with the notebook's check passed. The check tells you which dots are wrong, but never prints the answer.

Carry forward. The R peak is our landmark, and the RR interval is our first feature. An N label does not promise a clean signal: drift, noise bursts and inverted beats are all present in this dataset, and every step that follows has to survive them.

2 Measuring rhythm from ECG signals

Notebook: Section 2

What this section is for. Section 1 ended with a warning: the R peak is our landmark, but in a wearable recording it sits on a drifting baseline, inside noise bursts, and sometimes upside down, so picking the highest points of the signal is not enough. This section shows what we want *out* of such a signal before Section 3 builds the detector that gets it. It walks the whole route once, on three real recordings of the dataset, one of each rhythm: raw signal, the beats a detector finds in it, the intervals between those beats, and the handful of numbers that describe the rhythm. To do that without a detector of our own yet, the beats are found by an established library, NeuroKit2, which plays the part that your detector will play from Section 3 on. Everything after the beats is arithmetic, and you write that arithmetic in Task 2.

What people measure. Clinicians and wearables describe the rhythm of the heart by the *times* of its beats, not by the shape of the waves. Three quantities cover most of what is reported. The RR intervals are the gaps between consecutive R peaks. Their average gives the heart rate, the number every watch shows. How much they change from one beat to the next is heart rate variability (HRV), of which RMSSD is the most used measure and the one that separates atrial fibrillation from normal rhythm. Figure 3 shows all three on the N, A and O recordings of

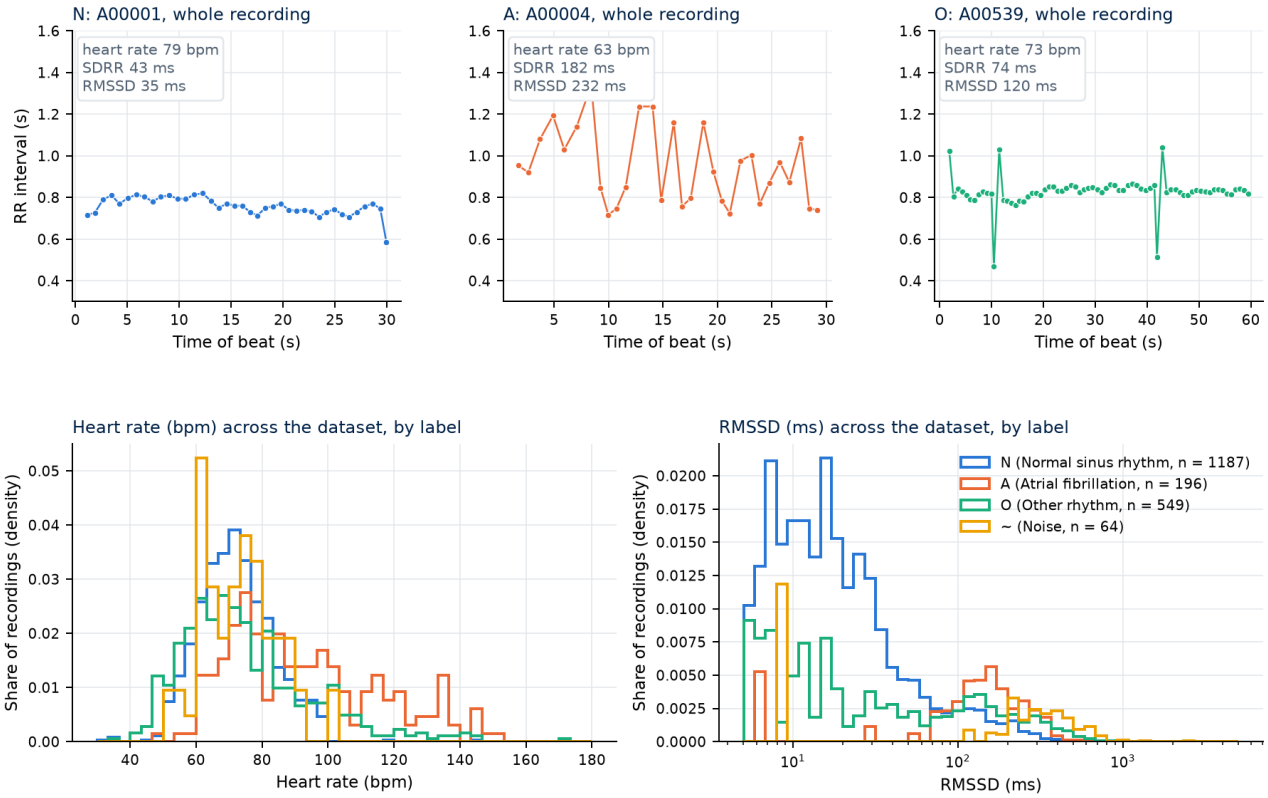


Figure 3: Top: the RR intervals of the `N`, `A` and `O` recordings of Figure 2, over the whole recording, with the heart rate, SDRR and RMSSD calculated from them. Bottom: heart rate and RMSSD (log axis) for every recording in the dataset, by label. Medians: `N` 71 bpm and 49 ms; `A` 89 bpm and 188 ms; `O` 71 bpm and 181 ms; `~` 73 bpm and 394 ms. Beats found by NeuroKit2.

Figure 2, and then heart rate and RMSSD for every recording in the dataset. The two numbers you compute here, heart rate and RMSSD, are the ones you will check and chase through the rest of the lab: every detector step in Section 3 is judged by how much it moves them, and Section 4 feeds them, with 23 relatives, to the classifier.

From beats to intervals. From K beats at sample indices $s_1, \ldots, s_K$ and sampling rate f_s:

$$t_k = s_k / f_s \qquad \text{beat times (s)}$$
$$RR_k = t_{k+1} - t_k \qquad \text{RR intervals (s), } K-1 \text{ of them.}$$

Plotting RR_k against time (Figure 3, top) is the quickest way to see a rhythm. The `N` recording is a nearly flat line that drifts slowly. The `A` recording jumps from one interval to the next. The `O` recording is flat except for one short interval followed by one long one: the premature beat and its pause.

Heart rate. Two versions are in use:

$$HR_k = 60/RR_k \qquad \text{instantaneous heart rate (bpm)}$$
$$HR_{\text{period}} = 60\,(K-1)/(t_K - t_1) = 60/\overline{RR} \qquad \text{rate over the period (bpm).}$$

The period rate is the number a wearable shows. All three recordings in the figure are in the resting range, 63 to 79 bpm, and the bottom-left panel shows the same across the dataset: the four labels overlap heavily. Heart rate alone does not tell the rhythms apart.

Heart rate variability. With the $N = K - 1$ intervals in milliseconds, the two most used time-domain features [4] are

$$\text{SDRR} = \sqrt{\frac{1}{N-1}\sum_{k=1}^{N}\left(RR_k - \overline{RR}\right)^2}, \qquad \text{RMSSD} = \sqrt{\frac{1}{N-1}\sum_{k=1}^{N-1}\left(RR_{k+1} - RR_k\right)^2}.$$

SDRR is the spread of the intervals around their mean; RMSSD is the typical change from one interval to the next. They answer different questions. A slow drift of the rate, as in the `N` recording, gives some SDRR (43 ms) and less RMSSD (35 ms). Abrupt changes, as in atrial fibrillation, give a lot of both (182 and 232 ms). One premature beat, as in the `O` recording, moves SDRR somewhat (74 ms) and RMSSD a lot (120 ms), because the short interval and the long one after it are two large successive differences. In the bottom-right panel RMSSD separates `N` from `A` well, with medians of 49 and 188 ms. `O` overlaps both, which is why it is the hardest class in Section 4, and `~` is largest of all, because wrongly placed beats look exactly like an irregular rhythm. Two cautions: call SDRR *SDNN* only when the intervals are confirmed to be normal-to-normal, and treat values from recordings this short as descriptive, not as a clinical assessment.

More of the same. NeuroKit2's `hrv_time` returns 25 time-domain HRV features, all variations on the ideas above (Table 1). Section 4 uses all of them, and they carry most of the classifier's performance.

Task 2. NeuroKit2 finds the beats of a recording, and the RR sequence is the differences between consecutive beat times. Implement `rhythm_features`, which turns an RR sequence into the heart rate, SDRR and RMSSD using the formulas above. The notebook runs it on the three recordings of Figure 3 and checks every value against NeuroKit2's own `hrv_time` on the same beats. *Deliver:* the function with the check passed, and the three-row table it prints.

What it measures	Features (NeuroKit2 names)
The typical interval, i.e. the rate	MeanNN, MedianNN, MinNN, MaxNN
Spread of the intervals	SDNN (our SDRR), CVNN, MadNN, MCVNN, IQRNN, Prc20NN, Prc80NN; SDANN1/2/5 and SDNNI1/2/5 over 1, 2 and 5-minute segments, undefined on recordings this short
Change from one interval to the next	RMSSD, SDSD, CVSD, pNN50 and pNN20 (share of successive differences above 50 and 20 ms), SDRMSSD
Shape of the RR histogram	HTI, TINN

Table 1: The 25 time-domain HRV features of `hrv_time`, grouped by what they measure (the `HRV_` prefix of each name is dropped).

Specification: `rhythm_features(rr_s)`

Given `rr_s`, a 1-D NumPy array of RR intervals in seconds, in time order: about 40 values for a 30 s recording, typically between 0.6 and 1.2 s, built from NeuroKit2's beats.

You may use NumPy only: the mean, the sample standard deviation, differences between neighbouring values, the square root.

Return a tuple of three floats `(heart_rate_bpm, sdrr_ms, rmssd_ms)`: $60/\overline{RR}$; the sample standard deviation of the intervals in ms; the square root of the mean of the squared successive differences in ms, squared before averaging.

Checked by NeuroKit2's `hrv_time` on the same beats, for three recordings, to six significant figures.

Carry forward. Heart rate is $60/\overline{RR}$; RMSSD is the root mean square of the successive differences, and you have coded both. Both need the beat times, and RMSSD needs every one of them to be right, which is where Section 3 starts.

3 Extracting rhythm features from noisy raw signals

Notebook: Section 3

In this section you will learn to extract two rhythm features, heart rate and RMSSD, from noisy raw ECG signals. The route is always the same: find the beats, then pass their times to the `rhythm_features` you wrote in Task 2. The work is finding the beats, and we go through it in four steps:

1. We use NeuroKit2, an established ECG library, as a reference beat detector (Section 3.1).
2. We implement a naive peak-finding beat detector: a threshold on the raw signal (Section 3.2).

3. We implement a detector that searches for the QRS complex instead of peaks, in three steps: filter, emphasise fast changes, locate the R peak (Section 3.3).

4. We add median imputation, which uses the rhythm itself to drop extra beats and fill in missed ones, and so corrects the HRV estimate (Section 3.4).

At steps 2 to 4 we measure how well our detector agrees with the NeuroKit2 reference: its beats, and the heart rate and RMSSD computed from them, across every recording in the dataset. The settings are teaching choices for 300 Hz data, not a validated clinical detector.

3.1 NeuroKit2: a reference beat detector

Notebook: 3.1

The NeuroKit2 reference. NeuroKit2 [1] is an open-source library used across research and industry to turn an ECG into beats and features, and its default pipeline (`ecg_clean`, then `ecg_peaks`) belongs to the same family as the detector you build in this section: filter, emphasise the fast QRS, threshold, place the beat. It is the *NeuroKit2 reference* for this section, as the notebook calls it: its beats for every recording, with no correction, and the heart rate and RMSSD from them (the circles in Figure 4). Every step you add is a step the library also takes, so closing the gap to it is how you find out what sits inside a library call, and where the practice has choices. Two things to keep in mind. The dataset labels rhythms, not beats, so there is no ground truth here: the reference is an established pipeline, not the truth, and a gap to it means *different*. And both detectors can make the same mistake, which no score can see.

The evaluation. In the notebook, each step is one short function that you write from its description (Tasks 4A to 4D), with its setting next to it and a self-check against a reference; a detector is the set of functions so far, kept in a dictionary: `steps_naive`, then `steps_4a = {**steps_naive, "filter": bandpass}`, and so on. `bench.score(name, steps)` runs a detector on every recording, prints three scores against the NeuroKit2 reference, enters it on a leaderboard and plots it next to the reference:

- **beat agreement**: beats paired with a reference beat within ±100 ms, as $2 \times \text{matched}/(\text{ours} + \text{reference})$; 1 when every beat pairs;
- **heart rate error**: |ours − reference| per recording, summarised by the MAE (mean absolute error, the typical error) and the P95 (the error only the worst 5% of recordings exceed);
- **RMSSD error**: the same, in ms.

3.2 A naive peak-finding detector

Notebook: 3.2, Task 3

The QRS is the fastest and usually the tallest event of each beat (Section 1), so a peak finder emphasises fast changes, thresholds the result, and places one beat in each region above the threshold, at the R peak. Section 3.3 builds it in three steps: filter out drift and fast noise; differentiate, square and integrate, so that each QRS gives one bump; locate the R peak inside each bump. On the recording in Figure 8 (Section 3.4) it finds 34 beats: 33 real ones, an artefact at 1.0 s, and none at 25.8 s, where the QRS was too small to cross the threshold.

Task 3: detector v1. The simplest peak finder is the threshold alone, and it is the first version of your detector. Write `naive_beats`, which takes an ECG signal and a threshold in millivolts and returns the sample index of each beat: one beat per region where the signal is above the threshold, at the region's highest sample. The notebook checks it on an invented signal and against a reference on your recording, shows your beats with the threshold next to NeuroKit2's, and turns them into heart rate and RMSSD with your `rhythm_features`: your first features extracted end to end. *Deliver:* the function with both checks passed, and the plot.

Specification: `naive_beats(signal, threshold)`

Given `signal`, a 1-D NumPy array of one ECG lead in mV, one value per sample at $f_s = 300$ Hz (9,000 samples for 30 s); `threshold`, one number in mV. The signal rises above the threshold at every QRS complex, and sometimes at a tall T wave or an artefact; each crossing is a run of consecutive samples above it, a *region*.

You may use NumPy only: comparisons, padding, differences between neighbouring values, the positions of non-zero values, slicing, and the position of a maximum.

Return a 1-D integer array of sample indices, one per region, at the region's highest sample, in increasing order; empty if nothing crosses. Time in seconds = index / f_s.

Checked by the invented signal `[0, 1, 3, 1, 0, 0, 2, 5, 2, 0, 1, 0]` with threshold 1.5, which must give `[2, 7]`; and a reference implementation on your recording.

Detector v1 on the benchmark. Your Task 3 rule, with one change: amplitude differs between people, electrodes and recordings, so the threshold is relative, 20% of the recording's maximum in its middle half, rather than a value in millivolts. Each region above the threshold gives one beat, at its highest point. On `A00001` this finds most beats; on `A00726` it finds 54 beats where there are 34, because P and T waves and a baseline swing also cross the threshold (Figure 4, and the top panel of Figure 5). Every later step keeps this threshold rule and changes one thing before it.

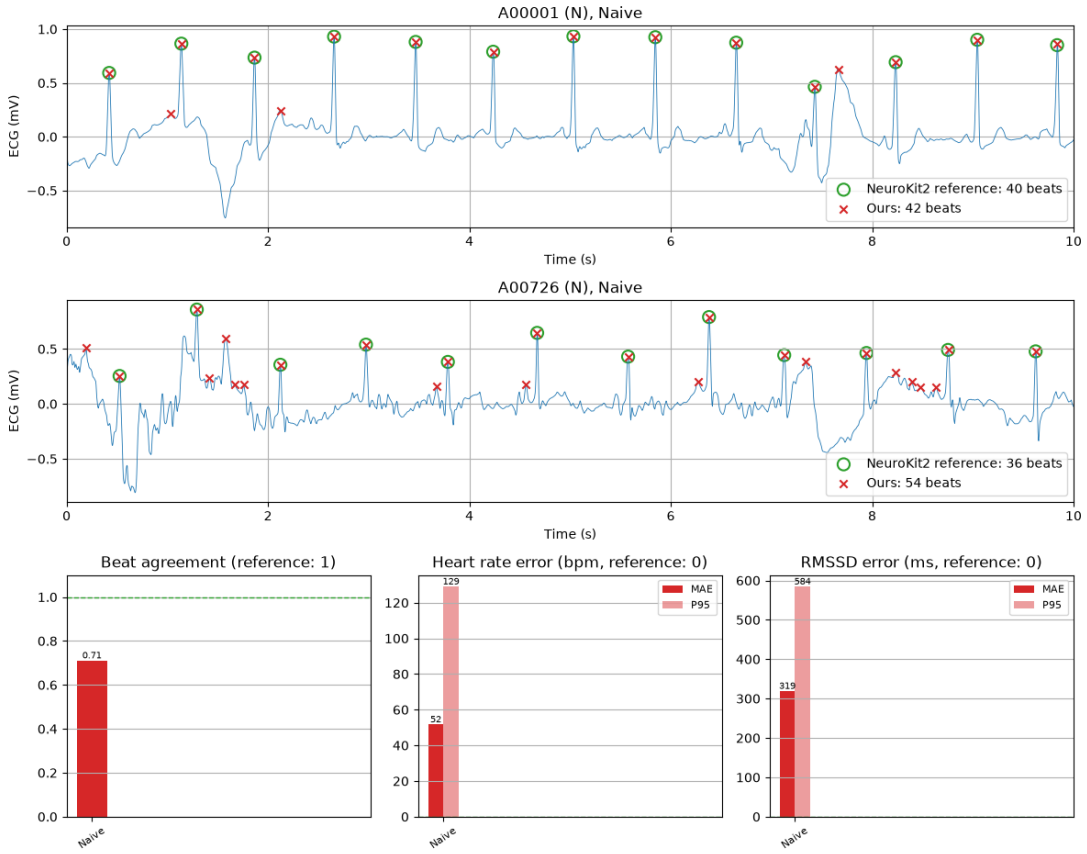


Figure 4: The naive detector (crosses) and the NeuroKit2 reference (circles) on `A00001` and `A00726`, and its scores across the dataset.

3.3 A beat detector that searches for the QRS complex

Notebook: 3.3, Tasks 4A–4C

In this subsection we develop a QRS-searching detector, detector v2. Instead of looking for tall points, it looks for the *rapid changes* that characterise the QRS complex: the sharpest edges in every beat, whatever their height or sign. It grows out of your naive detector v1 in three steps, in the style of the QRS detector of Pan and Tompkins [5]:

1. **Denoise** with a band-pass filter, so that baseline drift and fast noise do not cross the threshold (Task 4A).
2. **Differentiate, square and integrate**, so that the signal measures speed rather than height and each QRS complex becomes one bump (Task 4B).
3. **Locate the R peak** inside each bump, back on the ECG (Task 4C).

Each step is introduced by the *problem* the previous detector still has, the *idea* that addresses it, and the *caveats* it brings, and each is scored against the NeuroKit2 reference across all 1,996 recordings, so that you see what it buys and what it costs. The figures follow recording `A00726`

through every step. Getting close to an established library with a detector you built yourself is how you find out what the industry has been doing for decades to get features out of ECG signals.

3.3.1 Step 1: denoise with a band-pass filter

Notebook: Task 4A

Problem: the baseline drifts and fast noise rides on the signal; both can cross a threshold. *Idea:* keep the band where the ECG waves live, 0.5 Hz to 40 Hz, the bandwidth that monitoring ECGs have used for decades [6]: below 0.5 Hz sits baseline wander from breathing and electrode movement, above 40 Hz sit muscle noise and mains hum. A Butterworth filter applied forwards and backwards keeps the band without shifting any peak. *Caveats:* the band is an assumption; real features outside it are distorted, and a corrupted segment stays corrupted.

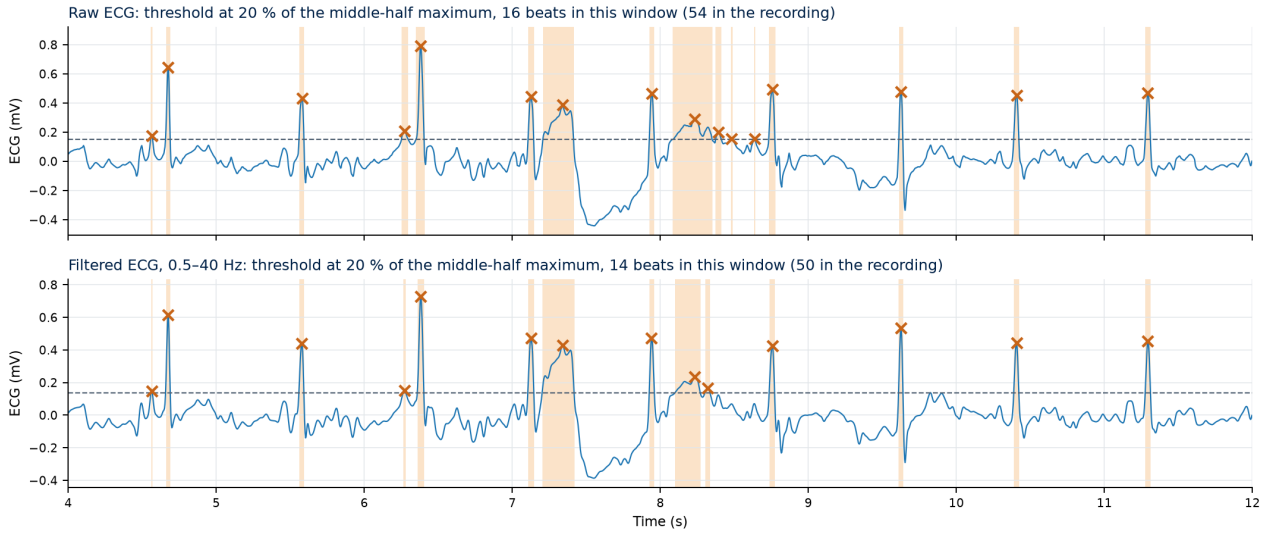


Figure 5: Step 1 on `A00726`, 4 to 12 s: the naive threshold rule on the raw ECG (top) and on the filtered ECG (bottom), with the regions above the threshold shaded and the beat each region gives. The baseline is flatter and two P-wave crossings are gone, but the swing at 7 s to 8 s still crosses the threshold.

What it bought. A cleaner signal, but not a better score: beat agreement 0.71 to 0.72, heart rate MAE 51.7 to 52.6 bpm. The filter does what it is for: the baseline is flatter and the small high-frequency noise outside the band is gone, which is why two P-wave crossings disappear in Figure 5. But the threshold is still a test of *height*, and the things that fool it most, T waves, slow swings and inverted QRS complexes, are ECG and live inside the band, so the filter cannot remove them. The filter comes first because of step 2: once height is turned into speed, any fast noise left in the signal is emphasised just as much as a QRS. On this dataset, which has

little fast noise, that is worth little (removing the filter from the full detector barely changes the scores); on a noisier wearable it would be worth more.

Task 4A. Write `bandpass`: design the filter with `butter` (order 4, the band 0.5 Hz to 40 Hz, `fs=FS`, second-order sections) and apply it with `sosfiltfilt`. *Deliver:* the function with the check passed, and its leaderboard line.

Specification: `bandpass(x)`

Given `x`, a 1-D array, the whole raw recording in mV at 300 Hz, with baseline drift and fast noise on it; the settings `low_hz` and `high_hz` (0.5 and 40) above the function.

You may use SciPy's `butter` and `sosfiltfilt`, imported at the top of the cell; their documentation gives the arguments.

Return a float array of the same length: the ECG through a fourth-order Butterworth band-pass between the two cutoffs, as second-order sections, applied forwards and then backwards so that no peak is shifted in time.

Checked by a reference implementation on your recording.

3.3.2 Step 2: differentiate, square, integrate

Notebook: Task 4B

Problem: height is a poor clue; a tall T wave can be as high as the R peak, and some QRS complexes point down. *Idea:* the QRS is different because it is *fast* (Figure 6). Differentiate, $d[n] = x[n] - x[n-1]$, to measure speed: large at the sharp QRS edges, small on slow waves. Square, so that both signs count and large values grow much more than small ones. Integrate with a centred moving average about one QRS wide (120 ms, 37 samples), merging the spikes of one beat into one bump. Only together do the three give the threshold one bump per beat, so they are one step. *Caveats:* a strong noise spike is emphasised just as much as a QRS, and the bump is smeared in time.

What it bought. Most of the gap: beat agreement 0.72 to 0.92, heart rate MAE 52.6 to 6.4 bpm. The RMSSD MAE also falls, 288 to 218 ms, but its P95 rises, 516 to 648 ms: on a few recordings a strong artefact now sets the threshold for the whole recording.

Task 4B. Write `differentiate` (the difference of neighbouring samples, with a leading zero so the length is unchanged) and `envelope` (square, then a centred moving average over the 120 ms window, rounded to an odd number of samples). *Deliver:* both functions with the checks passed, and the leaderboard line.



Figure 6: Step 2 on the same excerpt: filtered ECG, difference, squared difference, and the envelope with its threshold and regions. The swing that fooled step 1 is slow, so it almost vanishes in the difference; the envelope has one bump per beat, and the threshold now finds exactly the nine beats in the window.

Specification: `differentiate(x)` and `envelope(d)`

Given `x`, the filtered ECG, a 1-D float array in mV; then `d`, the difference signal, with positive and negative spikes at every QRS; the setting `integration_s` (0.12) above the function.

You may use NumPy only (for the envelope, a convolution with a window of equal weights, or cumulative sums).

Return `differentiate`: `d`, same length as `x`, with $d[n] = x[n] - x[n-1]$ for $n \geq 1$ and $d[0] = 0$. `envelope`: a non-negative array of the same length, at each sample the mean of d^2 over a window of `width` samples centred on it, where `width` is `integration_s` $\times f_s$ rounded to an integer and made odd (37 samples at 120 ms); beyond the edges the signal counts as zero.

Checked by the invented signal `[0, 1, 3, 1, 0]`, which must differentiate to `[0, 1,`

`2, -2, -1]`; and a reference envelope on your recording.

3.3.3 Step 3: locate the R peak on the ECG

Notebook: Task 4C

Problem: the envelope says roughly *when* a beat happens, but its maximum is smeared and sits after the R peak, and RR intervals are differences of these times. Occasionally one beat still gives two regions. *Idea:* in each candidate region, place the beat at the maximum of the filtered ECG (the minimum for an inverted recording, chosen only after inspecting the waveform), and keep no two beats closer than 250 ms, strongest first (Figure 7). *Caveats:* this rule can reject true fast beats or keep strong artefacts.

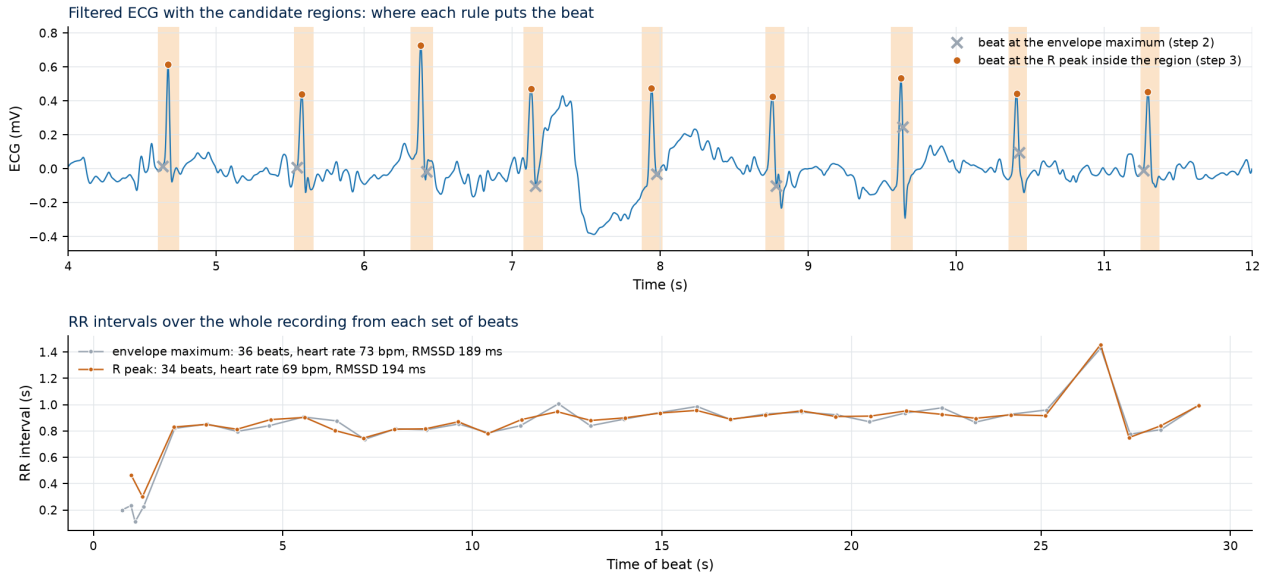


Figure 7: Step 3 on `A00726`. Top: in each region, the beat at the envelope maximum (step 2) sits about 40 ms after the R peak, where step 3 puts it. Bottom: the RR intervals from each set of beats over the whole recording. The grey line zigzags where the orange one is smooth; the two large spikes, the artefact at 1 s and the missed beat at 26 s, are left for median imputation.

What it bought. Beat agreement hardly moves (0.92 to 0.91: the envelope maximum was already within the 100 ms pairing tolerance), but the heart rate MAE falls from 6.4 to 5.1 bpm and the RMSSD MAE from 218 to 194 ms, because the intervals are now measured between R peaks and not between smeared maxima.

Task 4C. Write `locate`: one candidate per region, the highest sample of the filtered ECG (the lowest for negative polarity); then go through the candidates strongest first and keep one

only if it is at least 250 ms from every beat already kept. *Deliver:* the function with the checks passed, and the leaderboard line.

Specification: `locate(ecg, starts, stops)`

Given `ecg`, the filtered ECG in mV; `starts` and `stops`, integer arrays of equal length in time order, region k being `ecg[starts[k]:stops[k]]`, the regions disjoint, each meant to hold one beat but occasionally two regions on one beat. Inside the function `score` is defined: `ecg` for positive polarity, `-ecg` for negative, so the beat is always the highest sample of `score`. The settings `polarity` and `min_separation_s` (0.25) are above the function.

You may use `highest_in_regions(signal, starts, stops)`, supplied in the detector v1 cell, which returns the highest sample of each region; Python's sorting; NumPy.

Return a 1-D integer array of beat indices in time order: one candidate per region, with every candidate closer than `min_separation_s` $\times f_s$ samples to a *stronger* kept candidate removed. Going through the candidates strongest first guarantees that the stronger of two close candidates is the one kept.

Checked by an invented signal with peaks of height 5, 6 and 3 at samples 100, 150 and 500, which must give `[150, 500]`; and a reference implementation on your recording.

3.4 Median imputation: correcting the HRV estimate

Notebook: 3.4, Task 4D

Problem: after the three steps of Section 3.3 a few beats are still wrong: an artefact taken for a beat, or a small QRS that never crossed the threshold. Such an error barely affects the heart rate, but it does affect RMSSD, because one missed beat creates two large successive differences in a row. *Idea:* a peak-finding or QRS-searching detector is agnostic of rhythm. It detects each beat from local features only, which makes it sensitive to noise: an artefact right after a beat becomes two beats in rapid succession, and a small QRS becomes none. Yet a human heart beats with a specific rhythm, and the beats around a doubtful one say a lot about whether it is real. Median imputation adds that rhythm. The median RR interval is the recording's typical interval, and a few wrong beats barely move it. Going through the beats in time order, a beat closer than 0.6× the median RR to the last kept beat is dropped as an extra, and a gap longer than 1.6× the median RR is split evenly into round(gap / median RR) intervals, inserting beats. If the median rate is outside 30 bpm to 200 bpm, most beats are wrong and the median cannot be trusted, so nothing is changed. On the recording in Figure 8 it removes the artefact and fills the gap: RMSSD falls from 194 to 67 ms while the heart rate

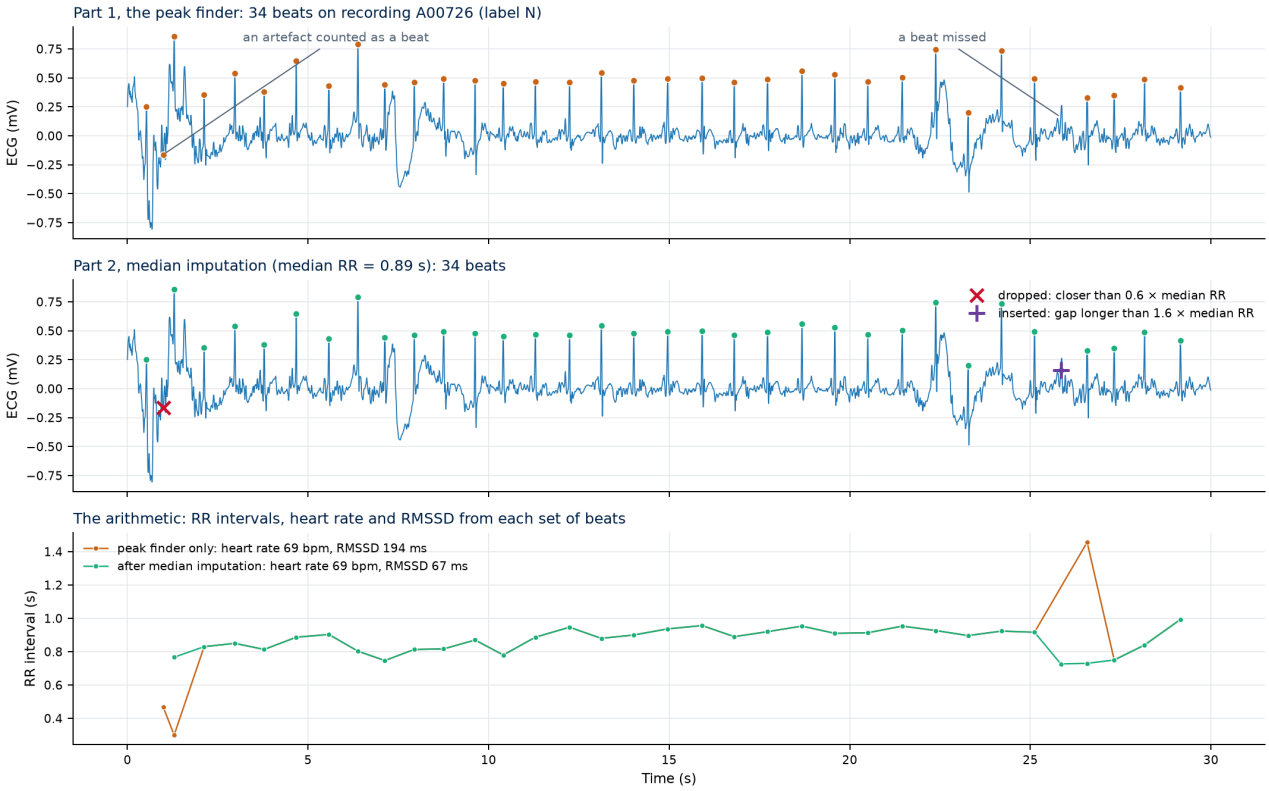


Figure 8: Median imputation on an `N` recording. Top: the peak finder's beats; it counts an artefact at 1.0 s and misses a beat at 25.8 s. Middle: the median imputation, with a median RR of 0.89 s, drops the first (closer than 0.6× the median RR to the previous beat) and inserts the second (a gap longer than 1.6× the median RR). Bottom: the RR intervals and the numbers from each set of beats. The heart rate does not change; RMSSD falls from 194 to 67 ms.

stays at 69 bpm. The reference is not corrected, so where it has an error of its own, correcting ours moves us away from it. *Caveats:* inserted beats are invented and evenly spaced, so they pull RMSSD down; a genuine premature beat, as in Section 1's `O` example, is "corrected" away; and in atrial fibrillation the irregularity is the physiology, not noise. The rule is a teaching version of two established ideas: QRS detectors search back for a missed beat when no beat has been found within about one and a half times the average RR interval [5, 7], and HRV editing identifies extra and missed beats as intervals that deviate from the local distribution and corrects them by merging or splitting intervals [8]. The current standard, used by Kubios and by NeuroKit2's `signal_fixpeaks`, classifies beats with time-varying thresholds on the successive differences [9]. The rule used here is the simplest member of that family, and that is why it was chosen for teaching: it fits in a few lines, and every decision it makes can be read off the RR plot. The factors 0.6 and 1.6 are teaching choices, not taken from any of these papers.

What it bought. The RMSSD MAE falls from 194 to 146 ms and its P95 from 604 to 393 ms; the heart rate P95 from 32 to 28.5 bpm. Beat agreement slips from 0.91 to 0.90: where the reference has a missed or extra beat of its own, our corrected beats no longer pair with it.

Task 4D. Write `correct`: the median RR of the beats, then the walk in time order with its two rules, drop when closer than 0.6 × the median, fill when further than 1.6 ×; the rate guard is supplied. *Deliver:* the function with the checks passed, and the leaderboard line.

Specification: `correct(beats)`

Given `beats`, a 1-D integer array of sample indices in time order from detector 4C, usually 30 to 60 per recording, occasionally with an extra one (an artefact) or a missing one (a small QRS); the settings `correct_short` and `correct_long` (0.6 and 1.6) above the function. The rate guard, the walk through the beats and the insertion of evenly spaced beats are supplied; you write the four quantities the rule needs.

You may use NumPy only.

Return a 1-D integer array in time order, the beats with every extra one dropped and every gap filled. `median_rr` is the median RR interval in samples; for each beat, `gap` (supplied) is its distance to the last kept beat; `too_close` is whether the gap is shorter than `correct_short` × `median_rr`; `too_far` is whether it is longer than `correct_long` × `median_rr`; `k` is the number of intervals the gap should hold, `gap / median_rr` rounded to an integer.

Checked by the invented beats `[0, 300, 600, 900, 1500, 1800, 1850, 2100]` (median RR 300), which must give `[0, 300, 600, 900, 1200, 1500, 1800, 2100]`; and a reference implementation on `A00726`.

4 Classifying rhythms with ECG features

Notebook: Section 4

In clinical practice, wearable ECGs are often used to detect heart problems that a clinic visit might miss. Atrial fibrillation is the main one: it often comes and goes, is often silent, raises the risk of stroke about fivefold [10], and can be treated once it is found. So people record short strips at home, by the million, and no cardiologist can read them all. In this section we use the two features of Section 3, heart rate and RMSSD, together with the other ECG features a library extracts, to predict the rhythm of each recording: `N`, `A`, `O` or `~`. That prediction is the triage: likely AF goes to a clinician, noise is recorded again, normal is filed. The label is a screening flag, not a diagnosis. The 2017 challenge that produced this dataset was set up for exactly this task, by the makers of such a device [2].

4.1 NeuroKit2 ECG features

Notebook: 4.1

From two features to the full set. In Section 3 you extracted two features, heart rate and RMSSD, with a detector you built. They are two of many. Once the beats are known, and the P, Q, S and T waves around each of them, a library can measure a whole family of numbers from the same recording (Table 2): how the RR intervals around each beat compare with one another; how far apart the waves of a beat are in time and in amplitude, which is the shape of the beat rather than its timing; the 25 HRV features of Table 1, which generalise RMSSD; and one number for the quality of the signal. Section 2 showed that RMSSD alone already separates `N` from `A` to a useful degree, but `O` overlaps both and `~` is scattered, so no single threshold will do. A classifier combines all 54 and draws the boundary itself. The route is interpretable, you can ask which features the classifier relies on, and it learns from about 1,400 labelled recordings. Its price is that every new problem needs new expertise and new features. Lab 2 asks what happens when nobody designs them.

How they are extracted. `extract_feature` runs the reference pipeline on a whole recording (`ecg_clean`, then `ecg_peaks`, no correction), locates the P, Q, S and T waves around each beat with `ecg_delineate`, and returns the 54 numbers of Table 2. It runs without visual review: that is what makes 2,000 recordings feasible, and also the risk: nobody checks the beats.

Family	Count	Features
R-peak timing	8	R amplitude (`a_R`); the RR intervals before, after and two after each beat (`RR0`, `RR1`, `RR2`) and their mean (`RRm`); the ratios `RR0/RR1`, `RR2/RR1`, `RRm/RR1`
Wave shape	20	time between wave peaks: `t_PR`, `t_QR`, `t_RS`, `t_RT`, `t_PQ`, `t_PS`, `t_PT`, `t_QS`, `t_QT`, `t_ST`; amplitude difference between wave peaks: `a_PQ`, `a_QR`, `a_RS`, `a_ST`, `a_PS`, `a_PT`, `a_QS`, `a_QT`, and the ratios `a_ST/a_QS`, `a_RS/a_QR`
Heart rate variability	25	**MeanNN** (the heart rate is 60 000/MeanNN), **RMSSD**, and the other 23 time-domain features of Table 1: SDNN, CVNN, MadNN, MCVNN, IQRNN, Prc20NN, Prc80NN, MedianNN, MinNN, MaxNN, SDANN1/2/5, SDNNI1/2/5, SDSD, CVSD, SDRMSSD, pNN50, pNN20, HTI, TINN
Signal quality	1	`SQI`, NeuroKit2's quality index

Table 2: The 54 features of `extract_feature`. Every one is computed per beat and averaged over the recording, except the HRV features and the SQI, which are computed over the whole recording. In bold, the two you computed in Section 3.

4.2 Training a RandomForest with the features

Notebook: 4.2, Task 5

We will train a RandomForest [11] of 100 trees on the 54 features and use it to predict the rhythm of unseen ECG recordings. The data are already prepared: the feature table (`ECG_feature.pkl`) was computed in advance for all 2,000 recordings, and the recordings are split 70% / 15% / 15% into training (1,400), validation and test sets with `random_state=42`, the same split as Lab 2. Because the classes are unbalanced, the evaluation metric is the *macro-averaged F1*: the mean of the four classes' F1 scores, so that each class counts equally. A reference training gives the scores in Table 3: accuracy 0.79 and a macro F1 of 0.64 on the test set.

Class	Precision	Recall	F1	Support
`N` normal sinus rhythm	0.85	0.92	0.88	173
`A` atrial fibrillation	0.71	0.73	0.72	30
`O` other rhythm	0.70	0.63	0.66	86
`~` too noisy	0.67	0.18	0.29	11
Macro average	0.73	0.62	**0.64**	300

Table 3: The RandomForest on the 300 test recordings. Accuracy is 0.79; the macro F1 of 0.64 (0.68 on the validation set) is the number Lab 2 compares against.

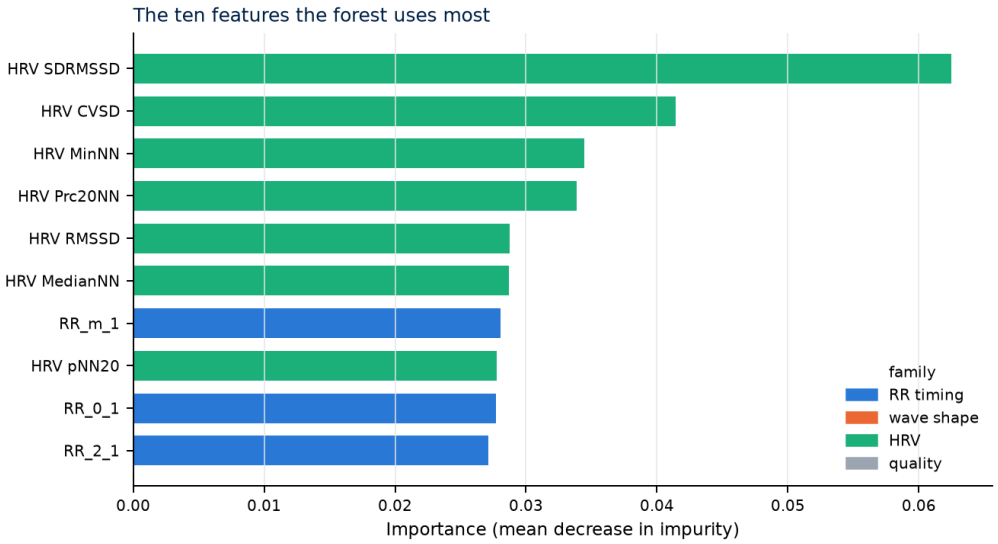


Figure 9: The ten features the forest uses most, coloured by family; all ten are rhythm features.

Analysing the result. Which features does the forest rely on (Figure 9)? Half of the importance sits in the 25 HRV features and another fifth in the 8 RR-timing features; the ten most used are all rhythm features, led by SDRMSSD, CVSD and RMSSD, which measure how much the intervals vary. That agrees with the definition of atrial fibrillation in Section 1: the rhythm is *irregularly irregular*, so what separates `A` is the intervals, not the waves.

Task 5. Write `train_forest`: a `StandardScaler` fitted on the training set only, and a `RandomForestClassifier` of 100 trees (`random_state=42`) fitted on the scaled training features; and `predict`: scale new features with the same scaler and return the model's predictions. The notebook checks your test-set predictions against a reference, prints the classification reports and plots the ten features the forest uses most. *Deliver:* both functions with the check passed; the test report; and your test macro F1, written down for Lab 2.

Specification: `train_forest(x_train, y_train)` and `predict(scaler, model, x)`

Given `x_train`, a 2-D float array of 1,400 rows (recordings) × 54 columns (features), unscaled, with ranges that differ by orders of magnitude between columns; `y_train`, an integer array of 1,400 class codes (0 to 3). For `predict`: the two fitted objects and `x`, $n \times 54$ unscaled features.

You may use `StandardScaler` and `RandomForestClassifier`, imported at the top of the cell; their documentation gives the arguments.

Return `train_forest`: `(scaler, model)`, a scaler fitted on `x_train` only (each column

centred and scaled by the training set's mean and standard deviation) and a forest of 100 trees with `random_state=42` fitted on the scaled `x_train` and `y_train`. `predict`: an integer array of n predicted class codes, from `x` scaled with the same scaler.

Checked by the reference model's predictions on the 300 test recordings, which yours must match exactly.

Carry forward. From a raw strip to a screening label, every step was designed by a person: the filter band, the envelope window, the median-RR rule, the 54 features. The result is interpretable and learns from 1,400 recordings, and it is only as good as the features and the beats under them. Lab 2 trains networks on the same recordings and the same split, with no designed features, and compares them with your macro F1.

Submitting your work

Save the notebook with your code and plots. Restart the kernel and run all cells to check that your work is reproducible, and keep your test macro F1 from Section 4 for Lab 2.

Acknowledgement. The learning sequence derives from the CWM Wearable Technology practical by Mauricio Villarroel, with detector material credited to Gari Clifford and Mauricio Villarroel [12]. The current 300 Hz implementation is a teaching adaptation, not a numerical reproduction of that detector.

Bibliography

[1] D. Makowski, T. Pham, Z. J. Lau, J. C. Brammer, F. Lespinasse, H. Pham, C. Schölzel, and S. H. A. Chen, "NeuroKit2: A Python toolbox for neurophysiological signal processing," *Behavior Research Methods*, vol. 53, pp. 1689–1696, 2021.

[2] G. D. Clifford, C. Liu, B. Moody, L.-w. H. Lehman, I. Silva, Q. Li, A. E. Johnson, and R. G. Mark, "AF classification from a short single lead ECG recording: the PhysioNet/Computing in Cardiology challenge 2017," in *Computing in Cardiology*, vol. 44, 2017.

[3] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[4] Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology, "Heart rate variability: standards of measurement, physiological interpretation and clinical use," *Circulation*, vol. 93, no. 5, pp. 1043–1065, 1996.

[5] J. Pan and W. J. Tompkins, "A real-time QRS detection algorithm," *IEEE Transactions on Biomedical Engineering*, vol. BME-32, no. 3, pp. 230–236, 1985.

[6] P. Kligfield, L. S. Gettes, J. J. Bailey, R. Childers, B. J. Deal, E. W. Hancock, G. van Herpen, J. A. Kors, P. Macfarlane, D. M. Mirvis, O. Pahlm, P. Rautaharju, and G. S. Wagner, "Recommendations for the standardization and interpretation of the electrocardiogram: Part I: The electrocardiogram and its technology," *Circulation*, vol. 115, no. 10, pp. 1306–1324, 2007.

[7] P. S. Hamilton and W. J. Tompkins, "Quantitative investigation of QRS detection rules using the MIT/BIH arrhythmia database," *IEEE Transactions on Biomedical Engineering*, vol. BME-33, no. 12, pp. 1157–1165, 1986.

[8] G. G. Berntson, K. S. Quigley, J. F. Jang, and S. T. Boysen, "An approach to artifact identification: application to heart period data," *Psychophysiology*, vol. 27, no. 5, pp. 586–598, 1990.

[9] J. A. Lipponen and M. P. Tarvainen, "A robust algorithm for heart rate variability time series artefact correction using novel beat classification," *Journal of Medical Engineering & Technology*, vol. 43, no. 3, pp. 173–181, 2019.

[10] P. A. Wolf, R. D. Abbott, and W. B. Kannel, "Atrial fibrillation as an independent risk factor for stroke: the Framingham study," *Stroke*, vol. 22, no. 8, pp. 983–988, 1991.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[12] M. Villarroel, "CWM wearable technology laboratory code." https://github.com/maurovm/cwm_laboratory_code. Revision 5103c09065bf8ea510461e096471456d0267e101.